



CENG 408

Innovative System Design and Development II
2025-2026 Spring

**MULTI-LABEL CHEST X-RAY DISEASE
CLASSIFICATION AND EXPLAINABILITY
WITH DEEP LEARNING**

METHODOLOGY

YAREN AKDOĞAN 202111046

SEZİN KÖROĞLU 202211046

İBRAHİM TAŞPINAR 202111072

CEVDET ALP ÖZGÜN 202211406

İPEK BAŞ 202211013

SALİH BARKIN AKKAYA 202211004

Abstract

This methodology presents an end-to-end system for multi-label thoracic disease classification from chest X-ray images, coupled with explainability and deployable software integration. The study uses the NIH ChestX-ray14 dataset and addresses key data challenges, including multi-label co-occurrence and severe class imbalance via a weighted learning strategy. To improve standardization and model robustness, images are filtered to Posterior–Anterior (PA) views and processed under multiple preprocessing pipelines (contrast/intensity enhancement, noise reduction, bone suppression, and frequency-domain filtering). All images are then standardized to a 224×224 , three-channel format to support transfer learning with common pretrained backbones. The core model employs a DenseNet-121 backbone augmented with Convolutional Block Attention Modules (CBAM) inserted after each dense block to strengthen sensitivity to subtle radiographic patterns while maintaining computational practicality. For interpretability, Grad-CAM is integrated to generate class-specific heatmaps highlighting image regions that drive predictions. The system is operationalized through a RESTful backend that orchestrates preprocessing, inference, and explainability, and persists traceable outputs (predictions, metadata, model versions, and explanation artifacts) in a MongoDB-based schema with external file storage for large media. A Flutter frontend consumes the API to deliver user-friendly upload, results visualization, and history retrieval.

TABLE OF CONTENTS

1. Dataset & Preprocessing Methodology	1
1.1. NIH ChestX-ray14	1
1.2. Dataset Filtering (PA view only)	1
1.3. Preprocessing Pipelines	2
1.3.1. Preprocessing Methods That Have Been Implemented.....	2
1.3.2. Preprocessing Methods That Will Be Implemented.....	2
1.4. Standardization for the Backend	3
2. Model Architecture And Training	4
2.1. DenseNet-121 Backbone	4
2.2. Convolutional Block Attention Module (CBAM)	4
2.2.1. Channel Attention Module	5
2.2.2. Spatial Attention Module	5
2.2.3. Overall Proposed Architecture	5
2.3. Model Explainability Using Grad-CAM	6
2.4 Model Training and Loss Functions.....	7
2.4.1. Binary Cross Entropy with Logits Loss (BCEWithLogitLoss)....	7
2.4.2. Asymmetric Loss (ASL)	7
2.4.3. Training Configuration And Hyperparameters.....	8
3. Backend & Database Methodology Implementation.....	8
3.1. Database Infrastructure with MongoDB Atlas.....	8
3.2 RESTful API Implementation with FastAPI.....	9
3.3 Frontend-Backend Collaboration.....	9
3.4 Explainability Workflow (GRAD-CAM)	10
4. Frontend	10
4.1. Frontend Interaction and User Interface Design	10
4.2. Secure Authentication and User Management	11
4.3. Doctor Dashboard and Patient Management	11
4.4. Analysis Workflow and Result Visualization	11
4.5. API Communication and Data Handling	12
5. References	13

1. Dataset & Preprocessing Methodology

1.1. NIH ChestX-ray14

Selection and the analysis of the dataset for this project, we selected the NIH ChestX-ray14 dataset, which is a standard for thoracic disease classification. We chose this dataset because it provides a sufficient amount of collection (112,120 chest X-ray images) which are labeled for 14 different conditions. During our analysis, we observed that the dataset has a multi-label structure. That means a patient could have multiple diseases at the same time.

We also identified a significant class imbalance issue. While common diseases like Infiltration had over 20,000 examples. Rare diseases like Hernia had very few samples. To prevent the model from ignoring these rare classes, we decided to create a weighted learning strategy for our training process. [1]

1.2. Dataset Filtering (PA view only)

Before starting the training process, we decided to filter the dataset based on the position of X-Ray views. The original dataset contains both Anterior-Posterior (AP) and Posterior-Anterior (PA) views. We chose to limit our dataset to PA views only, as they usually offer better standardization for deep learning models. We implemented a script that scanned the metadata and selected only the rows where the position of data was labeled as "PA". Based on this filtered list, we recreated the file directory and updated the training/testing split to ensure that no AP images were included in our final dataset.

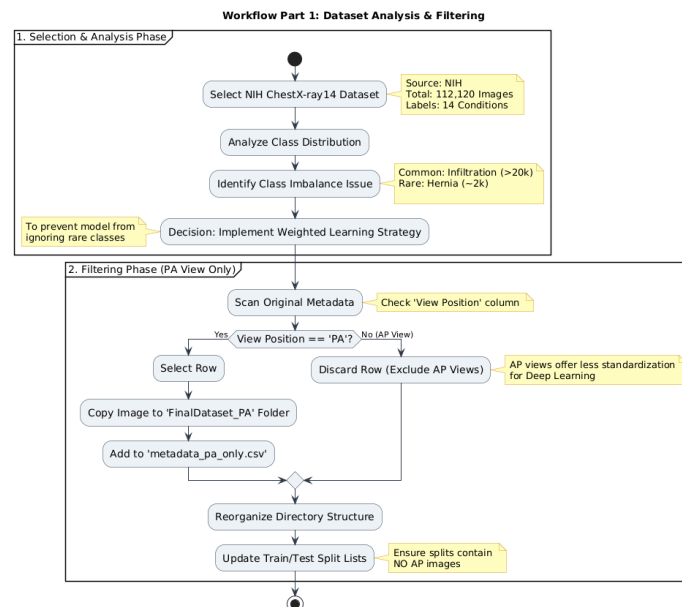


Figure-1: Dataset Analysis and Filtering Workflow Diagram

1.3. Preprocessing Pipelines

Since unprocessed X-ray images generally contain noise or varying lighting conditions, we designed four different preprocessing scenarios to improve the model's performance.

1.3.1. Preprocessing Methods That Have Been Implemented

These are the processing methods we have already implemented to see their effects on the results of our model training and to see whether or not they are suitable for the full application.

- **Intensity Enhancement:** In the first scenario, we focused on improving contrast. We applied Gamma Correction and Contrast Limited Adaptive Histogram Equalization (CLAHE) to make the details of lungs more visible. We also added an unsharp mask to make clear the edges [3],[5]. This method ultimately proved slightly less effective than simply using the images as they were found in the dataset and was discarded.

1.3.2. Preprocessing Methods That Will Be Implemented

These are the processing methods we are planning to implement after reaching a sufficient stage of development, where we can focus on minor improvements. We have applied these methods on a smaller scale to a limited number of images; however, we have not retrained the model or processed the full dataset using them.

- **Noise Reduction:** In the second scenario, we decided to reduce noise. We used a median filter to make the image smoother while preserving edges. Also, by CLAHE and a sharpening kernel, we highlighted the desired structures [3],[4].
- **Bone Suppression:** We experimented with a bone suppression technique to prevent the ribs from hiding lung abnormalities. We achieved this by subtracting a blurred version of the image from the original. That caused reducing the brightness of the bone structures and made the soft tissue clearer [3].
- **Frequency Domain Filtering:** Finally, we tested a method combining CLAHE with a Butterworth band-pass filter. This allowed us to emphasize specific structural details in the frequency domain while removing high-frequency noise [2],[3].

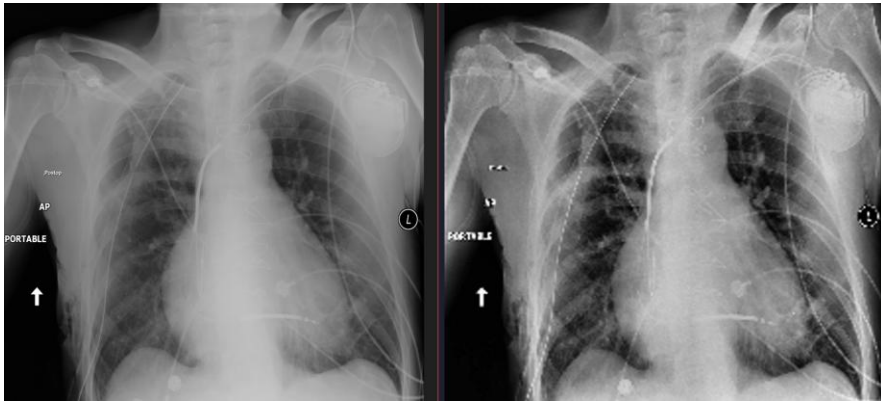


Figure-2: Intensity Enhancement

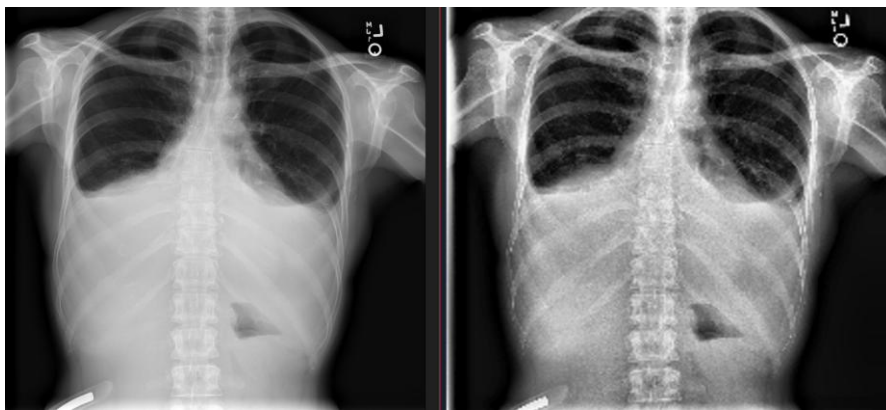


Figure-3: Noise Reduction



Figure-4: Bone Suppression

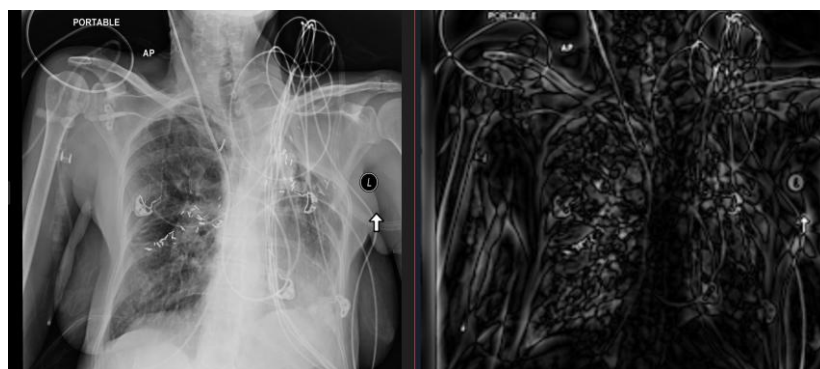


Figure-5: Frequency Domain Filtering

1.4. Standardization for the Backend

Independent of the chosen preprocessing strategy, we decided to put our processed dataset in a standard format. This method ensured the compatibility with our backend system. For every image in our dataset, we resized the resolution to 224x224 pixels. Even though the X-rays images in our dataset are originally in grayscale format we converted them into a 3-channel RGB format by repeating the grayscale channel three times. This decision was made to give us the opportunity to use pre-trained deep learning architectures (DenseNet, ResNet, etc.) that expect 3-channel input by default.

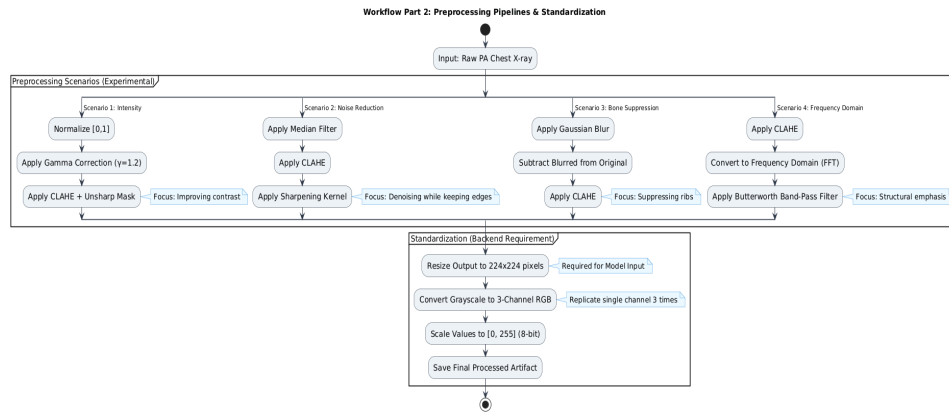


Figure-6: Preprocessing Pipelines and Workflow Diagram

2. Model Architecture And Training

2.1. DenseNet-121 Backbone

We have chosen DenseNet as the backbone of our model due to its proven success in medical image classification tasks. The dense connectivity technique used in DenseNet allows the outputs of previous layers to be utilized in all future layers within a dense block in the model, which mitigates the possible vanishing gradient issues we may encounter [6]. The pretrained ImageNet weights of DenseNet also allow for faster convergence and stable training. The segmented structure of DenseNet allows for the insertion of different modules between its dense blocks, which is perfect for our project. The dense blocks are made up of bottleneck layers with dense connectivity and make up a large part of the model. Bottleneck layers start with batch normalization and ReLU followed by a 1x1 convolutional layer, after which another batch normalization and ReLU, and end with one final 3x3 convolutional layer [6].

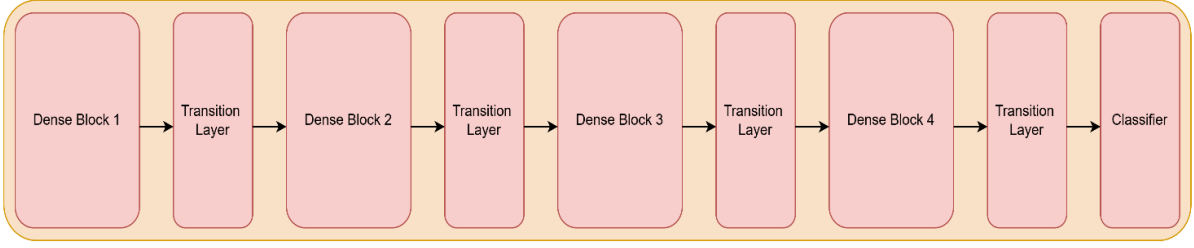


Figure-7: High Level Architecture of DenseNet-121

2.2. Convolutional Block Attention Module (CBAM)

CBAM is an attention module comprised of two smaller components called channel attention module and spatial attention module respectively [7]. Neither component affects the input or output sizes of the layers before and after them thus allowing for easy insertion into our model.

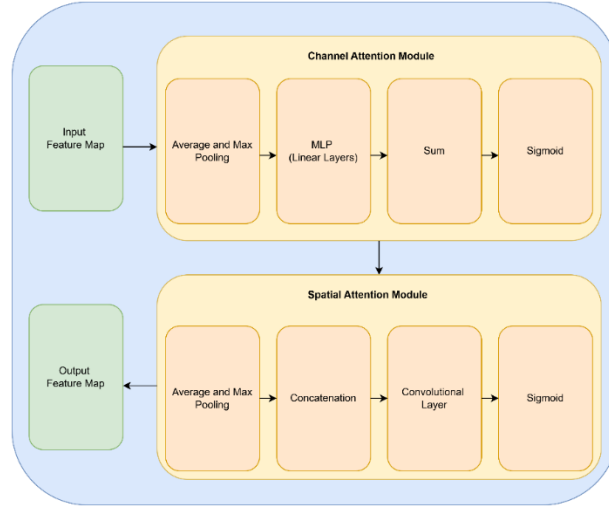


Figure-8: High Level Architecture of CBAM

2.2.1. Channel Attention Module

The channel attention module decides what features are important for a given task. The module applies average and max pooling to model the current features and passes them to an MLP. The sigmoid of the sum of the resulting features is multiplied with the current model features to update the feature map of the model [7].

2.2.2. Spatial Attention Module

The spatial attention module focuses on where important regions are on the feature maps. Similarly to the channel attention module, it applies average and max pooling across channels to generate maps that highlight where important features are. These feature maps are concatenated to make a feature map with two channels which is passed through a convolutional layer without changing its size while reducing the channel count back down

to one. The sigmoid of the resulting map is multiplied with the current feature map to highlight important spatial regions [7].

2.2.3. Overall Proposed Architecture

Our model adds a CBAM module after each dense block without changing the feature map size to avoid compatibility issues with subsequent dense blocks. By adding this module after each dense block, we aim to create a model that is more capable of detecting the small differences present in X-ray images without causing an excessive processing load on devices. Due to the precise nature of diagnosing thoracic diseases from X-ray images, we aim to optimize our model's confidence and interpretability. We are planning on using F1 and F1 macro as our metrics during development to evaluate the performance of our model.

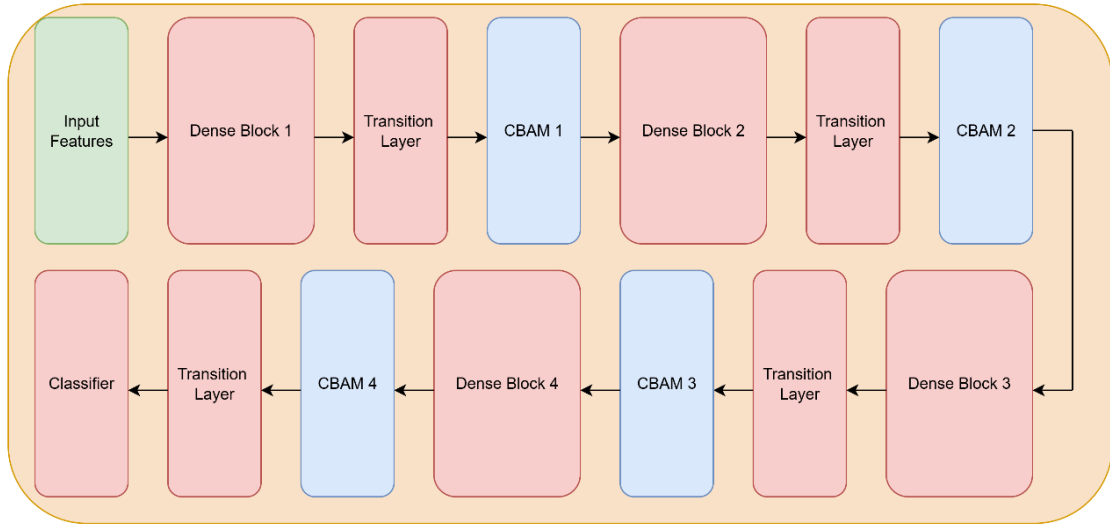


Figure-9: High Level Architecture of Our Model

2.3. Model Explainability Using Grad-CAM

Grad-CAM will be integrated into the model outputs to improve model interpretability. Grad-CAM will be used to visualize the areas that are relevant to the predictions our model makes by generating individual heatmaps over the original X-ray image for each possible disease class [8]. Through this technique, we aim to increase the trust of users by giving them the ability to see why the model came to a decision.

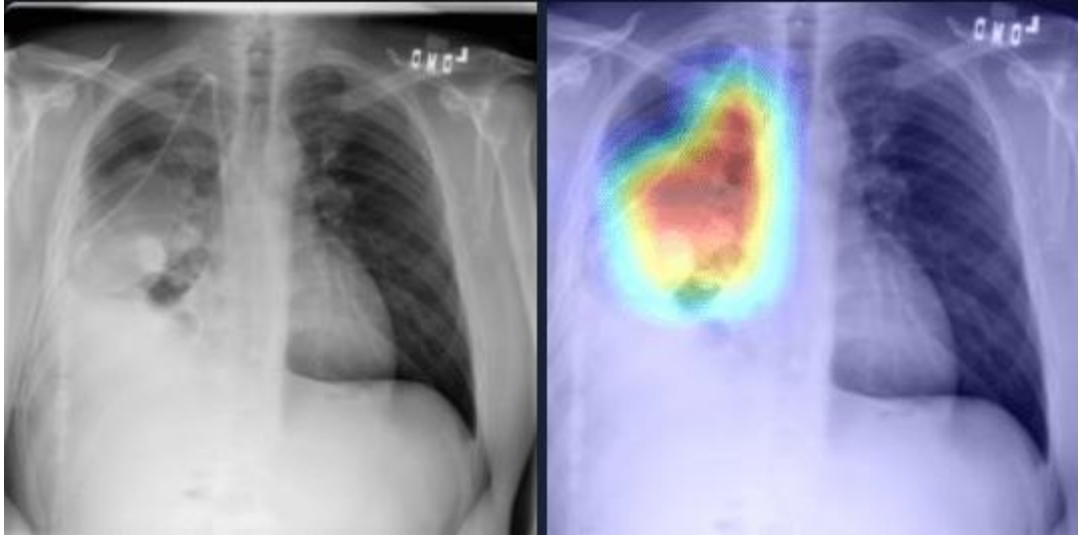


Figure-10: Original X-ray Image and The Grad-CAM Heatmap of The Same Image

2.4. Model Training and Loss Functions

The model training was designed for the multi-label nature of the dataset and to address the class imbalance across disease categories. Since each chest X-ray image may contain multiple pathologies simultaneously, the training methods and loss functions were selected with multi-label classification in mind.

2.4.1. Binary Cross Entropy with Logits Loss (BCEWithLogitLoss)

We used Binary Cross Entropy with Logits Loss as our initial baseline for our model. This loss function combines a sigmoid activation with binary cross entropy in a single equation and is widely used for multi-label classification tasks.

However, BCEWithLogitsLoss does not provide a way to get around the sample imbalance issue of our dataset. For the NIH ChestX-ray14 dataset, this can lead to the model performing much worse on rare disease classes.

2.4.2. Asymmetric Loss (ASL)

To overcome the class imbalance limitations of our baseline, we implemented Asymmetric Loss (ASL) as an alternative. ASL is specifically designed for imbalanced multi-label classification and can internally reduce the dominance of prevalent classes and allow for the model to learn less common ones.

The loss function applies asymmetric focusing to positive and negative samples, allowing the model to assign less importance to negative examples while emphasizing harder and more informative samples.

By emphasizing difficult examples and decreasing the weight of weighting common negatives, ASL improves the model's ability to detect subtle and rare thoracic diseases.

2.4.3. Training Configuration And Hyperparameters

Transfer learning was used by initializing the DenseNet-121 backbone with pretrained ImageNet weights. The model was trained using the Adam optimizer with a learning rate of $1e-4$.

Input image resolution was only lowered from 1024×1024 to 512×512 to preserve details that are important for detecting thoracic diseases. A batch size of 8 was used due to the large size of our dataset and the high resolution of our images.

Model performance was evaluated using F1 score and macro F1 score metrics, which provide a more accurate assessment in multi-label classification tasks.

3. Backend & Database Methodology Implementation

In this phase of the project, the architectural decisions made during CENG 407 have been fully implemented to establish a functional and scalable backend infrastructure. The system has been operationalized to ensure high availability and seamless data management for medical imaging workflows.

3.1. Database Infrastructure with MongoDB Atlas

For robust data management, a cloud-based cluster has been deployed using **MongoDB Atlas**, aligning with our strategy for handling flexible schema requirements.

- **Database Structure:** A dedicated database named "**Ceng408**" has been initialized within the cluster to manage project-specific data.
- **Collection Organization:** To ensure traceability, data is organized into three primary collections:
 - **Doctors:** Stores authentication details and profile information, including fields like `full_name`, `email`, and `role` (e.g., "doctor").
 - **Patients:** Manages patient demographics and links to their diagnostic history.
 - **Analyses:** Persists inference results, multi-label probability scores, and references to Grad-CAM heatmap paths.
- **Access Control:** To maintain system integrity, a strict administrative policy is enforced where only the two team members responsible for the backend have "Admin" privileges on the cluster.

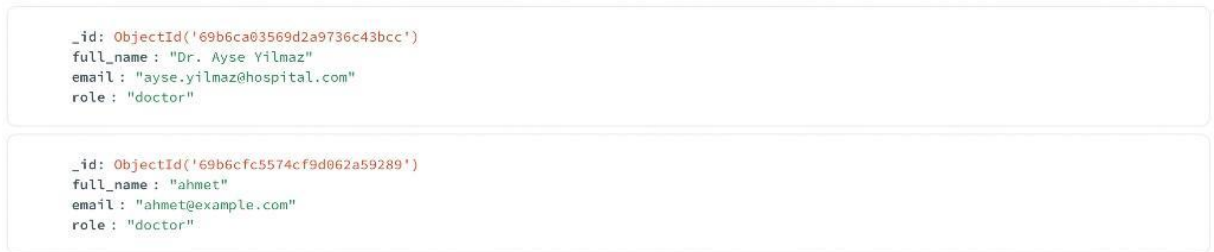


Figure -11: An example of a sample data.

3.2. RESTful API Implementation with FastAPI

The communication between the Flutter frontend and the backend is orchestrated using the FastAPI framework, which serves as the central coordinator for the machine learning pipeline.

Performance and Validation: FastAPI was selected for its high performance and native support for asynchronous operations, critical for handling image preprocessing and model inference.

API Endpoints: Key endpoints such as POST /register, POST /analyze, and GET /results have been implemented to handle client requests and initiate the analysis workflow.

Data Interchange: All exchanges are standardized using JSON, ensuring platform independence and lightweight communication between the server and the Flutter interface.

3.3. Frontend-Backend Collaboration

The database schema and API definitions were designed in close coordination with the frontend team. This integration ensures that when a user uploads a chest X-ray via the Flutter interface, the backend handles the multi-step workflow—including MongoDB storage, preprocessing, and model inference—before returning a structured response to the UI.

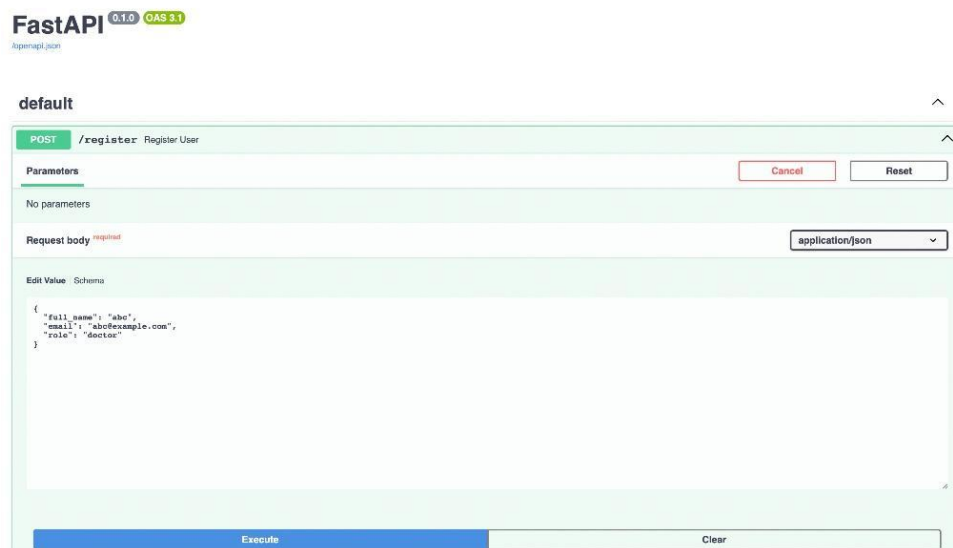


Figure-12: Testing the POST /register endpoint on Swagger UI using user registration data.

3.4. Explainability Workflow (Grad-CAM)

The explainability component using **Grad-CAM** has been integrated directly into the backend infrastructure as a core capability. This integration allows clinicians to visualize the specific regions of the chest X-ray that contribute most to the model's diagnostic predictions.

- **Operational Logic:** Grad-CAM is treated as a separate, on-demand action within the backend to manage the additional computational load effectively.
- **Data Management:** Once a heatmap is generated, it is saved as an image file on the server's storage layer, and its corresponding metadata and URL are persisted in the **Analyses** collection in MongoDB.
- **Traceability:** This workflow enables the system to store a permanent record of how a specific model version produced a particular visual explanation, supporting later clinical review and auditability.

```
1  {
2    "inference_id": "INF_7721",
3    "patient_data": {
4      "image_id": 1,
5      "path": "/storage/xray_001.png"
6    },
7    "model_info": {
8      "name": "DenseNet121",
9      "version": "v1.2"
10   },
11   "results": {
12     "probabilities": {
13       "Effusion": 0.82,
14       "Atelectasis": 0.15
15     },
16     "threshold": 0.5,
17     "final_labels": ["Effusion"]
18   },
19   "explainability": {
20     "method": "Grad-CAM",
21     "heatmap_url": "/outputs/cam_001.png"
22   },
23   "timestamp": "2025-12-18T21:20:00Z"
24 }
```

Figure -13: An example of a JSON document from our database. It shows how we store the model's probability scores and the file paths for the Grad-CAM outputs in one place.

4. Frontend

4.1. Frontend Interaction and User Interface Design

The frontend of the ChestXplain system is developed as a native Android application using Kotlin and XML-based layout architectures. The user interface is designed to provide professional and intuitive experience for clinicians, prioritizing clarity in medical data visualization. While initial planning considered a Flutter-based client[13,14], the current implementation leverages native Android components to ensure high performance and seamless integration with the mobile operating system's hardware, such as the camera for X-ray capture.

4.2. Secure Authentication and User Management

The application implements a secure entry point through LoginActivity and RegisterActivity modules.

- **Registration (RegisterActivity):** New users can create accounts by providing their full name, email, and password. The system performs client-side validation, ensuring all fields are populated and that the password meets a minimum length of 6 characters. To enhance user experience, the registration button includes scale-based click animations for tactile feedback.
- **Login (LoginActivity):** Existing medical staff can authenticate using their credentials. The interface features "link-style" navigation managed via native UI flags.
- To ensure that only authorized medical professionals can use the system, an admin approval mechanism is implemented. When a new user submits a registration request, a notification is sent to the administrator. The administrator reviews the request and decides whether the user is a verified medical professional.
- Based on this evaluation, the administrator can either approve or reject the registration request. Only approved users are stored as active users in the system and are allowed to access the application features.
- This approach prevents unauthorized individuals from accessing sensitive medical functionalities and ensures that the system is restricted to verified doctors.
- Additionally, once a user is approved and successfully logs in, authentication tokens are used to maintain secure communication between the frontend and backend. These tokens are included in API requests to ensure that only authenticated users can access protected resources.

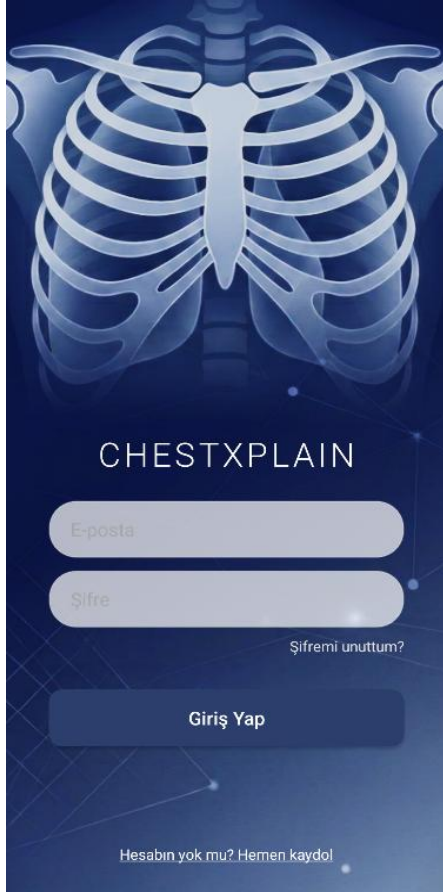


Figure-14: Login Interface with

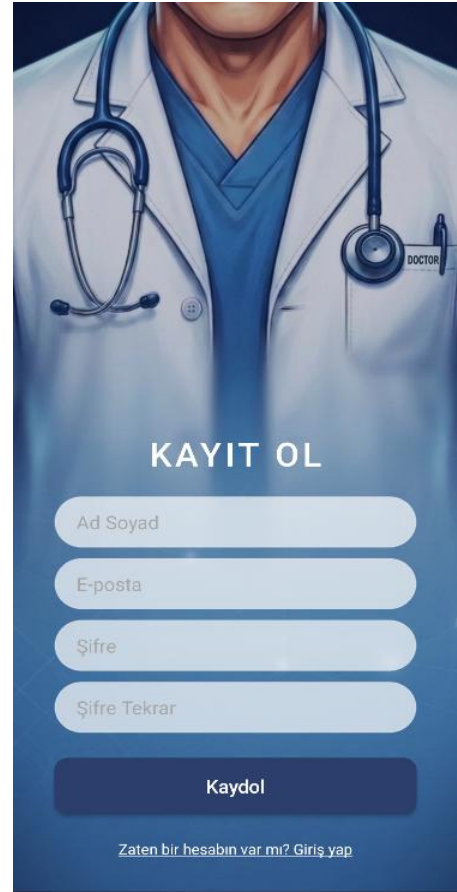


Figure-15: User Registration Interface

4.3. Doctor Dashboard and Patient Management

The MainActivity serves as the central hub for clinicians. This dashboard is designed to facilitate a streamlined diagnostic workflow adhering to software quality requirements for medical systems [15]:

- **History Retrieval:** Doctors can access a list of past analyses and patient diagnostic history stored in the **MongoDB** "Patients" and "Analyses" collections.
- **Image Selection:** The flow allows users to select a chest X-ray image from the device gallery or capture a new one via the camera.
- **Pre-Analysis Confirmation:** Once selected, the image is displayed on-screen for the user to confirm the correct orientation and quality before initiating the backend request.

4.4. Analysis Workflow and Result Visualization

When the analysis is triggered, the frontend manages the data transmission and result presentation:

- **Asynchronous Communication:** The selected image is sent to the **FastAPI** backend via multipart/form-data requests. A loading indicator is displayed to inform the user that the deep learning model (DenseNet-121 + CBAM) is processing the data.
- **Probability Mapping:** The system receives a JSON response containing multi-label probability scores for the 14 thoracic conditions. These results are mapped to internal data structures and displayed alongside their confidence values.
- **Grad-CAM Integration:** For model explainability, the application retrieves Grad-CAM heatmaps[8,10] generated on the backend. These visual explanations highlight the specific lung regions that influenced the model's prediction, fostering clinical trust.

4.5. API Communication and Error Handling

The application ensures system robustness through comprehensive data handling and feedback mechanisms:

- **JSON Standardization:** All prediction results and metadata are transferred in standardized JSON format (ECMA-404) [16] to maintain platform independence.
- **Input Validation:** The frontend warns users if no image is selected or if an unsupported file type is detected before the request is sent.
- **Network Resilience:** In the event of network failures or server-side errors, the application displays meaningful HTTP status-based error messages [17] and provides a "retry" mechanism to maintain the clinical workflow.

5. References

- [1] National Institutes of Health, “NIH Chest X-rays Dataset,” Kaggle, [Online].
Available: <https://www.kaggle.com/datasets/nih-chest-xrays/data>.
Accessed: 19 Dec. 2025.
- [2] A. Giełczyk, A. Marciniak, M. Tarczewska & Z. Lutowski, “Pre-processing methods in chest X-ray image classification,” *PLOS ONE*, vol. 17, no. 4, e0265949, Apr. 2022.
Available: <https://journals.plos.org/plosone/article?id=10.1371/journal.pone.0265949>
Accessed: 19 Dec. 2025.
- [3] IEEE Xplore, “Article (document no. 10955260),” [Online].
Available: <https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=10955260>.
Accessed: 19 Dec. 2025.
- [4] “Median Filter – an overview,” ScienceDirect Topics, Elsevier, [Online].
Available: <https://www.sciencedirect.com/topics/engineering/median-filter>.
Accessed: 19 Dec. 2025.
- [5] N. F. Sahib and Z. A. Hashim, “Contrast Image Enhancement by Gamma Correction,” *Computer Engineering and Intelligent Systems*, vol. 9, no. 7, 2018.
Available: <https://scispace.com/pdf/contrast-image-enhancement-by-gamma-correction-3upr34p2k1.pdf>.
Accessed: 19 Dec. 2025.
- [6] G. Huang, Z. Liu, L. van der Maaten, and K. Q. Weinberger, “Densely Connected Convolutional Networks,” *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017, [Online].
Available: <https://arxiv.org/abs/1608.06993>
Accessed: 20 Dec. 2025.
- [7] S. Woo, J. Park, J.-Y. Lee, and I. S. Kweon, “CBAM: Convolutional Block Attention Module,” *Proceedings of the European Conference on Computer Vision (ECCV)*, 2018, [Online].
Available: <https://arxiv.org/abs/1807.06521>
Accessed: 20 Dec. 2025.
- [8] R. R. Selvaraju, M. Cogswell, A. Das, R. Vedantam, D. Parikh, and D. Batra, “Grad-CAM: Visual Explanations from Deep Networks via Gradient-Based Localization,” *IEEE*

International Conference on Computer Vision (ICCV), 2017, [Online].

Available: <https://arxiv.org/abs/1610.02391>

Accessed: 19 Dec. 2025.

[9] X. Wang et al., “ChestX-ray8: Hospital-scale chest X-ray database and benchmarks on weakly-supervised classification and localization of common thorax diseases,” *CVPR*, 2017.

[10] I. E. Ihongbe et al., “Evaluating Explainable Artificial Intelligence (XAI) techniques in chest radiology imaging through a human-centered lens,” *PLOS ONE*, 2024.

[11] J. Irvin et al., “CheXpert: A Large Chest Radiograph Dataset with Uncertainty Labels and Expert Comparison,” *AAAI*, 2019.

[12] J. Han et al., “BO-CLAHE enhancing neonatal chest X-ray image quality for improved lesion classification,” *Scientific Reports*, 2025.

[13] Flutter Team. Flutter Documentation.

<https://docs.flutter.dev>

[14] Google Developers. Flutter Architectural Overview.

<https://docs.flutter.dev/resources/architectural-overview>

[15] ISO/IEC 25010:2011. Systems and software Quality Requirements and Evaluation (SQuaRE).

[16] ECMA International. (2017). The JSON Data Interchange Syntax (ECMA-404).

[17] Mozilla Developer Network (MDN). HTTP Overview and Status Codes.